fn solve_nb_x_from_mean

Michael Shoemate

March 16, 2026

Proves soundness of `fn solve_nb_x_from_mean` in `mod.rs`.

1 Hoare Triple

Precondition

`mean` > 1 and `eta` ≥ 0.

Pseudocode

```
1  def solve_nb_x_from_mean(mean: float, eta: float):
2      lo = 0.0
3      hi = 1.0
4      while expected_nb_mean_from_x_hi(eta, hi) > mean:  #
5          hi *= 2.0
6          if not hi.is_finite():  #
7              raise "failed to solve the repetition distribution parameter from mean"
8      while True:  #
9          mid = lo + (hi - lo) / 2.0  #
10         if mid == lo or mid == hi:  #
11             return hi
12         if expected_nb_mean_from_x_hi(eta, mid) > mean:  #
13             lo = mid
14         else:
15             hi = mid
```

Postcondition

Theorem 1.1. For any inputs satisfying the precondition, `solve_nb_x_from_mean` either returns `Err(e)`, or returns `Ok(out)` where

$$\texttt{expected_nb_mean_from_x_hi(eta, out)} \leq \texttt{mean}.$$

Moreover, `out` is the final value of the upper bisection endpoint `hi`.

2 Proof

Assume the precondition.

Lemma 2.1. `solve_nb_x_from_mean` only returns `Err(e)` if some call to `expected_nb_mean_from_x_hi` propagates `Err(e)` or if the explicit finite guard on line 6 fails.

Proof. These are the only fallible operations in the pseudocode. □

Lemma 2.2. Whenever control reaches the top of the bisection loop on line 8,

$$\texttt{expected_nb_mean_from_x_hi(eta, hi)} \leq \texttt{mean}.$$

Proof. When the expansion loop terminates normally, the guard on line 4 is false, so the current `hi` satisfies the inequality. Inside the bisection loop, `hi` is updated only in the `else` branch, and there it is set to `mid` only after checking that `expected_nb_mean_from_x_hi(eta, mid)` $\leq$ `mean`. Thus the property is preserved from one iteration to the next. □

Lemma 2.3. If the function does not return `Err(e)`, then the bisection loop eventually reaches line 10 and returns.

Proof. At the start of the bisection loop, `lo` and `hi` are finite and satisfy `lo < hi`. Each non-terminating iteration sets exactly one endpoint to the midpoint on line 9, so the new interval is a strict subinterval of the previous one and still has finite endpoints.

Because `f64` has only finitely many representable values in any bounded interval, this strict shrinking cannot continue indefinitely. Therefore after finitely many iterations there is no representable `f64` strictly between `lo` and `hi`. At that point the midpoint rounds to one of the endpoints, so the test on line 10 succeeds and the function returns. □

Lemma 2.4. If `solve_nb_x_from_mean` returns `Ok(out)`, then

$$\texttt{expected_nb_mean_from_x_hi(eta, out)} \leq \texttt{mean},$$

and `out` is the final value of `hi`.

Proof. The only successful return is on line 10, which returns `hi`; therefore `out` is exactly the final value of the upper endpoint.

By Lemma 2.2, the displayed inequality holds at the start of every bisection iteration. The convergence test does not modify `hi`, so it still holds at return. □

Proof of Theorem 1.1. Holds by Lemma 2.1, Lemma 2.3, and Lemma 2.4. □